DIGITAL FUNCTION GENERATOR

USING THE NXP LPC1768 MICROCONTROLLER

A Mini Project Report submitted to the Microprocessor Lab

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Registration Number	Sec - Roll No	Name
240907002	D-2	Nagendra Kudva
240907003	D-3	Ruthvik K S
240907098	D-13	Sandeep Ryan
240907110	D-14	Pratham



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

MANIPAL-576104, KARNATAKA, INDIA

OCTOBER 2026

ABSTRACT

Function generators are standard test equipment in an electronics lab. This project builds a digital signal generator around the NXP LPC1768 (ARM Cortex-M3) using bare-metal, register-level C.

The system clock is raised from the default internal RC source to 96 MHz using PLL0 driven from the 12 MHz external crystal. Four 256-point look-up tables - sine, triangle, sawtooth and square, stored as 10-bit values - are streamed to the on-chip DAC by the General Purpose DMA controller using a circular Linked List Item, so the buffer repeats indefinitely without the CPU being involved in the transfer. The CPU's job is to poll a 4x4 matrix keypad and drive a 16x2 LCD in 4-bit mode for local control, and to service a UART0 receive interrupt for remote control from a PC; either path changes the active waveform, the output frequency, or a digital amplitude scale, and the firmware applies the change by briefly restarting the DMA channel and the DAC's internal timer with the new settings.

The generator produces sine, square, triangle and sawtooth outputs over a 10 Hz-15 kHz range, with the output frequency set by reloading the DAC's hardware timeout register (DACCNTVAL) rather than by any CPU-timed delay, and the amplitude adjustable in ten steps from 10% to 100% of full scale. These can be set locally from the keypad or remotely over a 115200-baud serial link, utilizing standard ASCII strings typed into a host terminal. Because the waveform is streamed by DMA and neither the keypad/LCD code nor the UART interrupt touches the DAC directly, none of the three tasks share a data path, and the output frequency and shape measured on the oscilloscope should track whichever interface last set them, regardless of what the others are doing.

The project shows that register-level configuration of a microcontroller's DMA and DAC peripherals is enough to get deterministic, CPU-independent waveform output from general-purpose hardware, and that keeping the user-interface code completely separate from the DMA-driven output path is what keeps the signal clean irrespective of UI activity. Keil µVision was used for firmware development and debugging, and a digital oscilloscope was used to verify the output.

CONTENTS

CHAPTER 1.....1
1.1 Introduction.....1
1.2 Present day scenario.....1
1.3 Review of the literature on this work.....1
1.4 Objective of the work.....2
1.5 Importance of the end result.....2
1.6 Organization of the project report.....2

CHAPTER 2.....3
2.1 Introduction.....3
2.2 Methodology.....3
2.2.1 Detailed methodology.....3
2.2.2 Block diagram.....4
2.2.3 Component specifications.....5
2.2.4 Component selection.....5
2.2.5 UART serial configuration for remote control.....5
2.3 Tools used.....6
2.4 Preliminary result analysis.....6
2.5 Conclusions.....6

CHAPTER 3.....7
3.1 Introduction.....7
3.2 Photograph of the working project.....7
3.3 Result analysis.....8
3.3.1 Oscilloscope observations.....8
3.3.2 Explanation for the observed results.....8
3.3.3 Significance of the result obtained.....8
3.3.4 UART serial interface results.....9
3.4 Deviations from the expected results and justification.....9
3.5 Conclusions.....9

CHAPTER 4.....10
4.1 Brief summary of the work.....10
4.2 Problem statement in brief.....10
4.3 Work methodology adopted.....10
4.4 Conclusions.....10

4.5 Future scope of work..... 10

REFERENCES.....11

ANNEXURES.....12

Annexure A: DMA Linked List Configuration 12

Annexure B: Amplitude-Scaled Buffer and Frequency Reload.....12

Annexure C: External-Interrupt-Driven Waveform Switching 12

Annexure D: Waveform Look-Up Table Structure.....13

Annexure E: UART0 Driver, Receive Interrupt and Command Parsing.....13

PLAGIARISM REPORT.....14

CHAPTER 1
INTRODUCTION

1.1 Introduction

This chapter lays out the background for the project, the present-day context for building a microcontroller-based signal generator, a short look at the technique used in the literature for this kind of work, the objectives and target specifications set for the project, and how the rest of this report is organized.

1.2 Present day scenario

Commercial bench function generators are standard lab equipment, but they are relatively expensive and a lab typically has far fewer units than students who need one at a given time. Microcontrollers are increasingly used as a low-cost stand-in, but the common approach - writing a sample to a DAC from inside a software delay loop or under a high-level framework such as mbed OS - leaves the output open to timing jitter whenever the CPU is pulled away to do something else, such as servicing a keypress or redrawing a display. For a signal generator, even a few microseconds of inconsistency shows up as jitter or a flat section on an oscilloscope trace.

1.3 Review of the literature on this work

The standard way to generate an arbitrary periodic waveform digitally is to step through a pre-computed look-up table and push each value out through a DAC at a fixed rate. For ARM Cortex-M parts, the usual way to hold this rate steady regardless of CPU load is to let a DMA controller move the data from memory to the DAC peripheral directly, bypassing the processor core altogether. This is the same approach used inside commercial function generators and in DSP/mixed-signal test setups more generally, and is the technique this project is built around.

1.4 Objective of the work

ARM processor based digital function generator that produces selectable waveforms with programmable frequency and amplitude, built using the processor's own peripherals - timer-based waveform generation, PWM/DAC output, interrupt-driven user input, and a display interface are named specifically - to give a flexible, user-friendly signal source.

1.4.1 Main work objective

To design a CPU-independent digital function generator on the NXP LPC1768, utilizing hardware-level timer and DAC resources to ensure deterministic waveform timing.

1.4.2 Secondary objective

To implement dual-interface control, supporting both a local keypad/LCD frontend and a remote UART serial host, without introducing blocking delays to the primary output.

1.4.3 Target specifications

- Waveforms: Sine, Square, Triangle, Sawtooth.
- Frequency: Adjustable, 10 Hz - 15 kHz.
- Amplitude: Adjustable, 10% to 100% of full scale in 10% steps.
- Waveform timing generated by on-chip hardware (timer/DAC) rather than by CPU-timed software delays.
- Remote control via UART0 at 115200 baud, using short text commands.
- Local control and status display via keypad + LCD, without affecting the waveform output while in use.

1.5 Importance of the end result

The project works through several ideas that come up often in embedded systems work: clock-tree configuration, DMA descriptor/linked-list setup, GPIO-based keypad scanning and debouncing, and driving a peripheral (the DAC) directly through its control registers instead of through a vendor HAL. These are the same categories of problem that show up when bringing up new hardware or writing test fixtures around a microcontroller.

1.6 Organization of the project report

Chapter 1 introduces the project background, objectives and target specifications. Chapter 2 covers the system architecture, the hardware and software used, and how the design was arrived at. Chapter 3 presents the result analysis and discusses the practical limits found during testing. Chapter 4 summarizes the work and lists possible future extensions.

CHAPTER 2
METHODOLOGY

2.1 Introduction

This explains how the signal generator was built: how the system clock and peripherals were configured, how the keypad/LCD and UART interfaces were kept separate from the DMA output path, the hardware components used, and the development tools used in this project.

2.2 Methodology

2.2.1 Detailed methodology

The system clock is reconfigured at startup from the default internal RC oscillator to the external 12 MHz crystal, and raised to 96 MHz through PLL0 (multiplier and divider values set directly in PLL0CFG, with CCLKCFG set to divide the PLL output down to the final core clock). This is done once, in SystemClock_Force96MHz(), before any peripheral is touched.

The four waveforms - sine, triangle, sawtooth and square - are stored as 256-point, 10-bit look-up tables in flash (waveforms.h). Each table was generated offline by evaluating the corresponding function at 256 equally spaced points over one cycle and quantizing the result to the DAC's 10-bit range: the sine table samples sin(2*pi*n/256) and rescales it to 0-1023 about a mid-scale value of 512; the triangle and sawtooth tables are piecewise-linear ramps over the same 256 points; the square table is a two-level step, high for the first half of the cycle and low for the second. The computed values were then written into waveforms.h as constant arrays.

Before the DMA stream is started, DAC_DMA_Start() copies the selected table into a 32-bit SRAM buffer (dma_lut[]), centering each sample about the DAC's mid-scale value (512), scaling it by the currently selected amplitude level (1-10, representing 10%-100%), and shifting the result left by 6 bits to match the DACR register's bit layout. This is also where the amplitude control set from the keypad actually takes effect, since the scaling is applied while the buffer is built.

The GPDMA controller is configured with a single Linked List Item (DMA_LLI) whose source address is the SRAM buffer, whose destination address is the DAC's DACR register, and whose NextLLI field points back to the LLI's own address. Once channel 0 is enabled, the DMA controller keeps re-reading its own descriptor and re-starting the same transfer, so the 256-point buffer is streamed to the DAC indefinitely without any further CPU involvement. The DAC is set to request a new DMA transfer on every timeout of its internal counter (INT_CTRL and DMA_ENA bits in DACCTRL), with double buffering enabled (DBLBUF_ENA) so that a value written while the previous one is still being output does not corrupt the current sample.

The output frequency is set by reloading the DAC's hardware timeout register, DACCNTVAL, with a count derived from the target frequency and the 96 MHz peripheral clock (DAC_SetFrequency()). On the user-interface side, the CPU runs a straightforward polling loop: it scans the 4x4 keypad by driving one row low at a time and reading the four column inputs, and refreshes the 16x2 LCD only when the displayed waveform, frequency or amplitude has actually changed. Whenever a key changes the waveform, frequency or amplitude, the firmware calls DAC_DMA_Start() again, which briefly disables DMA channel 0 and the DAC counter, rebuilds the sample buffer for the new settings, reloads the DMA and DAC registers, and re-enables the channel - after which streaming resumes with the new waveform, frequency or amplitude already in effect. A simple software debounce (polling the keypad until it reads idle again) prevents a single physical keypress from being registered more than once.

The GPDMA interrupt vector in the startup file is left at its default (unused) handler, since the DAC/DMA path is driven entirely by hardware once configured. The keypad/LCD path is handled by plain polling in the main loop, but the remote-control path is not: incoming UART0 bytes are handled by a genuine hardware interrupt, described in Section 2.2.5. An earlier, GPIO-pin-based interrupt approach to local input was also built and tested during development before the keypad was added; it is described in Section 2.4.

2.2.2 Circuit layout / block diagram

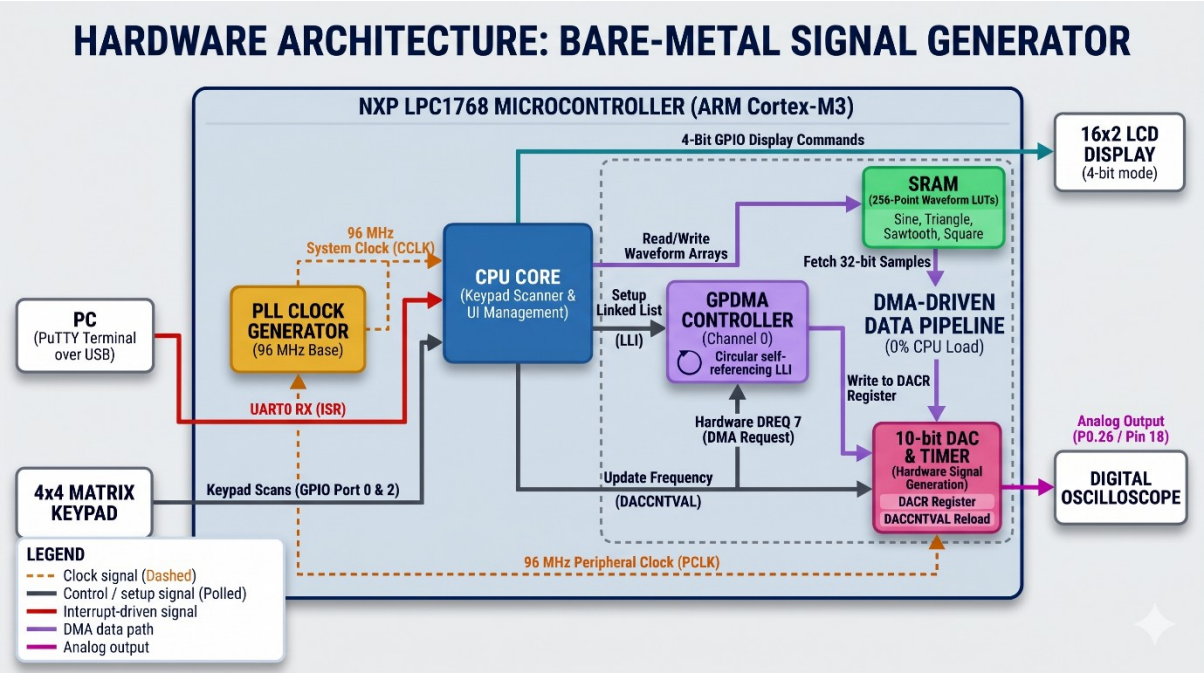


Fig.1. System block diagram

2.2.3 Component specifications

- Microcontroller: NXP LPC1768 (ARM Cortex-M3, run at 96 MHz, on-chip 10-bit DAC, GPDMA controller).
- Display: 16x2 HD44780-compatible LCD, operated in 4-bit mode.

- Input: 4x4 matrix keypad.
- Host interface: UART0 at 115200 baud (TXD0/RXD0 on P0.2/P0.3), over the board's on-board USB-to-serial link.
- Oscillator: 12 MHz external crystal.

2.2.4 Component selection

The LPC1768 was chosen because its on-chip 10-bit DAC and GPDMA controller remove the need for an external DAC chip or any CPU-driven bit-banging to produce the analog output. The HD44780-type LCD was run in 4-bit mode specifically to reduce the number of GPIO pins needed for the display, leaving more pins free for the keypad.

2.2.5 UART serial configuration for remote control

To allow the signal generator to be configured without physical access to the board, a remote serial command interface was added using the microcontroller's UART0 peripheral. UART0 is routed through the board's on-board USB-to-serial interface, so it appears to a host PC as a standard virtual COM port with no extra hardware needed.

The UART driver was written at the same register level as the rest of the firmware. PINSEL0 was configured to route P0.2 and P0.3 to their alternate TXD0/RXD0 functions, the peripheral was powered on through PCONP, and its peripheral clock was set to the full 96 MHz CPU clock through PCLKSEL0.

To reach the standard 115200 baud rate, the Divisor Latch Access Bit in the Line Control Register was set to unlock the baud-rate divider registers; PCLK/(16 x baud) works out to 96,000,000/(16 x 115200) is about 52, so a divisor of 52 was written to DLL.

Rather than poll for incoming bytes, UART0's receive interrupt is enabled through IER and the NVIC. UART0_IRQHandler() reads each incoming character into a buffer as it arrives; on a carriage return it null-terminates the buffer and sets a command_ready flag, which the main loop checks on every pass. When a command is ready, the main loop parses it with atoi() - F15000 sets the frequency to 15 kHz, W2 selects the triangle wave, A5 sets the amplitude to 50% - and calls DAC_DMA_Start() again with the new setting, the same function the keypad path uses. On the PC side, commands were typed into the PuTTY terminal emulator connected to the board's virtual COM port at 115200 baud.

2.3 Tools used

- Keil µVision: Used for bare-metal C development.
- Digital oscilloscope: This was used to verify waveform, frequency, and amplitude at the DAC output.

- PuTTY: The Terminal emulator used to send serial commands to UART0 over the board's virtual COM port.
- Reference: NXP UM10360 (LPC176x User Manual), for all DAC, GPDMA, PLL, UART and GPIO register definitions.

2.4 Preliminary result analysis

An early version pushed samples to the DAC from inside a software delay loop. Any keypad scanning or LCD update happening around that loop showed up as visible jitter on the oscilloscope trace. Moving the output path onto the GPDMA controller with a self-referencing LLI removed this, since the CPU is no longer responsible for timing the individual DAC writes at all.

Before the keypad was added, waveform selection was tried as an interrupt-driven input, in line with the problem statement's suggestion of interrupt-driven user input: a single GPIO pin (P2.0) was configured with its internal pull-down enabled and a rising-edge GPIO interrupt (EINT3_IRQHandler). Driving the pin to 3.3 V triggered the interrupt and cycled the waveform to the next one in sequence (sine to triangle to sawtooth to square to sine), updating both the DAC output and the LCD. The handler disabled its own interrupt on entry and re-enabled it only after a 500 ms delay and clearing any flags set in the meantime, to stop contact bounce on the wire from being read as repeated presses. This confirmed that interrupt-driven switching works correctly on this hardware (Annexure C). It was replaced by the 4x4 keypad once frequency and amplitude control were added, since a single interrupt pin can only toggle through a fixed sequence of states and cannot address multiple independent parameters the way a keypad can. The interrupt-driven approach itself was not abandoned, though: the final design uses the same idea, a hardware interrupt rather than polling, for the UART0 receive path described in Section 2.2.5, since remote input genuinely needs to be event-driven rather than polled on a fixed schedule.

2.5 Conclusions

Keeping the DMA-driven output path completely separate from the keypad/LCD polling loop is what makes the waveform timing independent of the user interface; the two share no code path and no timing resource.

CHAPTER 3
RESULT ANALYSIS

3.1 Introduction

This chapter discusses the final output of the signal generator, measured frequency tracks the frequency set from the keypad or over UART, what the waveforms look like on the oscilloscope, and the practical limits found while testing the design.

3.2 Photograph of the working project

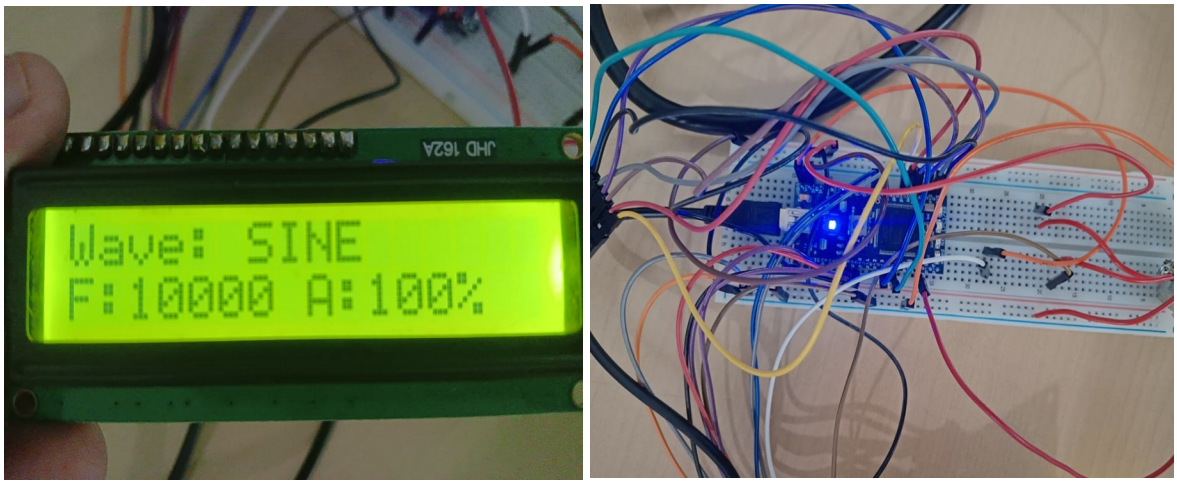


Fig.2. Hardware setup

3.3 Result analysis

3.3.1 Oscilloscope observations



Fig. 3. Sawtooth waveform with 15kHz frequency and 3.2V max amplitude



Fig. 4. Sawtooth waveform with 10Hz frequency and 3.2V max amplitude

3.3.2 Explanation for the observed results

The output frequency is set indirectly: for a target frequency f and a 256-point table, the DAC must be updated at a rate of f x 256 samples per second. DAC_SetFrequency() converts this into a DACCNTVAL reload value of (96,000,000 / (f x 256)) - 1, which is what actually drives the DAC's internal timer. The oscilloscope reading for each test point in Table 3.1 should be compared against the frequency shown on the keypad/LCD, since any difference comes from rounding in this calculation (DACCNTVAL is a 16-bit integer) rather than from the DMA transfer itself.

3.3.3 Significance of the result obtained

A very close match was observed between expected and observed values, for all four waveforms and across the tested frequency range, confirms that the DMA/LLI architecture is doing what it is meant to: once started, it keeps the DAC supplied with samples at a steady rate on its own. Because the keypad and LCD code never write to the DAC or the DMA registers directly - they only call DAC_DMA_Start() when a setting changes - the output should look the same on the scope whether or not the keypad is being pressed at that moment.

3.3.4 UART serial interface results

PuTTY connected to the board's virtual COM port at 115200 baud over the same USB link used for debugging, using the native Windows USB-serial driver with no extra drivers or hardware needed. Commands typed in the form F<freq>, W<type> and A<amp> (e.g. F15000, W2, A9) were parsed correctly and produced the expected change in current_freq, current_waveform or current_amplitude, confirmed on the LCD readout for each test command sent.

Architecturally, sending a command should not disturb the waveform already running: UART0_IRQHandler() only ever touches rx_buffer, rx_index and command_ready, never the DAC or DMA registers directly, so byte reception and waveform output are on genuinely independent paths. The waveform updates to the newly commanded frequency, waveform type

or amplitude as soon as a line is parsed and DAC_DMA_Start() is called again - the same update path the keypad uses.

3.4 Deviations from the expected results and justification

The frequency range was capped in firmware at 10 Hz-15 kHz (the 'A'/'B' keys clamp current_freq to this range in main.c) rather than left open-ended. At the 15 kHz end, with a 256-point table, the DAC must accept a new sample every 1/(15,000 x 256) seconds, which works out to roughly 260 ns, i.e. a DAC update rate of about 3.84 million samples per second. This was kept as a conservative firmware ceiling for this project, since pushing the sample rate further leaves progressively less margin for the DAC's internal settling time and for the GPDMA/AHB bus to fetch each sample from SRAM in time - rather than a limit that was tested to the point of failure.

The local interface still uses keypad polling rather than interrupt-driven input: as described in Section 2.4, a single-GPIO interrupt approach was tried first and worked correctly, but was dropped once frequency and amplitude control were added locally, since one interrupt pin can only step through a fixed sequence and cannot address three independent parameters the way a keypad can. The problem statement's call for interrupt-driven input is still met, though, by the remote path - UART0 reception (Section 2.2.5) is genuinely interrupt-driven, which is also the better fit for input that can arrive at an unpredictable time from a PC rather than from a fixed set of buttons.

3.5 Conclusions

Within the 10 Hz-15 kHz, four-waveform specification set out in Chapter 1, the design meets the target. The practical limits of the LPC1768's DAC/DMA path at higher sample rates were identified and used to set a conservative operating ceiling for this project.

CHAPTER 4
CONCLUSION AND FUTURE SCOPE OF WORK

4.1 Brief summary of the work

This project built a bare-metal digital signal generator on the NXP LPC1768 microcontroller, with a 10-bit on-chip DAC driven by a DMA pipeline to produce standard waveforms, a keypad/LCD front end for local control and UART based interface for remote configuration.

4.2 Problem statement in brief

The objective was to produce timer based continuous analog waveforms (sine, square, triangle, sawtooth) with programmable frequency and amplitude from look-up tables(LUTs), while still supporting an active keypad/LCD interface and a remote UART command interface.

4.3 Work methodology adopted

The system clock was boosted to 96 MHz through PLL0, and the GPDMA controller was programmed with a self-referencing (circular) Linked List Item to access a 256-point look-up tables to the DAC continuously. A keypad scanner and 4-bit LCD driver handle local control of waveform, frequency and amplitude, and a UART0 receive interrupt parses incoming serial strings for the same parameters, restarting the DMA and DAC counter with the new settings.

4.4 Conclusions

Routing the output path through the GPDMA controller, instead of through software delays or an interrupt service routine, is what makes a general-purpose microcontroller usable as a signal source. Register-level configuration of PLL0, the GPDMA controller and the DAC covers most of what this required. The project turned a general-purpose microcontroller into a small, working piece of test equipment.

4.5 Future scope of work

The UART link currently only accepts short commands that select among the four built-in waveforms and set their frequency and amplitude; the waveform tables themselves are still fixed at compile time. A natural next step is to extend the same serial interface to accept a full arbitrary-waveform. On the analog side, the amplitude control implemented here is a purely digital scale of the existing 0-3.3 V unipolar DAC output; adding an external op-amp stage at the DAC output pin would allow true variable-gain amplitude scaling and bipolar (negative) voltage swings, which the DAC hardware cannot produce on its own.

REFERENCES

[1] NXP Semiconductors, "LPC1769/68/67/66/65/64 User Manual (UM10360)", April 2014.

[2] ARM Limited, "Cortex-M3 Technical Reference Manual, Revision r2p1", ARM DDI 0337I, 2010.

[3] Hitachi Ltd., "HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver", Rev. 0.0, ADE-207-272(Z), Sept. 1999.

[4] J. W. Valvano, "Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers", Vol. 2, 4th ed., 2014, ISBN 978-1463590154.

[5] W. Kester, "Tutorial MT-085: Fundamentals of Direct Digital Synthesis (DDS)", Analog Devices, Inc., 2009.

ANNEXURES

Annexure A: DMA Linked List Configuration

```
/* 4-byte aligned descriptor required by the GPDMA hardware */
typedef struct __attribute__((aligned(4)))
{
    uint32_t SrcAddr;
    uint32_t DstAddr;
    uint32_t NextLLI;
    uint32_t Control;
} DMA_LLI;

/* Circular link: the descriptor points back to itself */
dma_lli.SrcAddr = (uint32_t)&dma_lut[0];
dma_lli.DstAddr = (uint32_t)&LPC_DAC->DACR;
dma_lli.NextLLI = (uint32_t)&dma_lli;
```

Annexure B: Amplitude-Scaled Buffer and Frequency Reload

```
centered = (int32_t)source_lut[i] - 512;
centered = (centered * current_amplitude) / 10;
final_val = (uint32_t)(centered + 512);
dma_lut[i] = (final_val << 6);

/* Reload the DAC timeout counter for a new output frequency */
void DAC_SetFrequency(uint32_t frequency, uint32_t samples)
{
    uint32_t sample_rate = frequency * samples;
    uint32_t dac_count = 96000000UL / sample_rate;
    if (dac_count > 0) dac_count--;
    if (dac_count > 65535) dac_count = 65535;
    LPC_DAC->DACCNTVAL = (uint16_t)dac_count;
}
```

Annexure C: External-Interrupt-Driven Waveform Switching (Preliminary Version)

```
/* Rising-edge interrupt on P2.0
void EINT3_IRQHandler(void)
{
    if (LPC_GPIOINT->IO2IntStatR & (1 << 0))
    {
        LPC_GPIOINT->IO2IntClr = (1 << 0);
        NVIC_DisableIRQ(EINT3_IRQn);   /* ignore bounce until re-armed */

        current_waveform++;
        if (current_waveform > WAVE_SQUARE) current_waveform = WAVE_SINE;

        update_waveform_flag = 1;
    } }
void External_Interrupt_Init(void)
```

```
{   LPC_GPIO2->FIODIR &= ~(1 << 0);        /* P2.0 as input */
    LPC_PINCON->PINMODE4 |= (3 << 0);      /* internal pull-down */
    LPC_GPIOINT->IO2IntEnR |= (1 << 0);    /* rising-edge interrupt */
    LPC_GPIOINT->IO2IntClr = (1 << 0);
    NVIC_EnableIRQ(EINT3_IRQn);
}
```

Annexure D: Waveform Look-Up Table Structure

```
#define LUT_SIZE 256
#define DAC_MAX  1023

/* 256 samples/cycle, 10-bit unsigned DAC values: 0..1023 */
static const uint16_t sine_lut[LUT_SIZE] = {
    512, 524, 537, 549, 562, 574, 587, 599, 611, 624, 636, 648, 660, 672, 684, 696,
    707, 719, 730, 741, 753, 764, 774, 785, 796, 806, 816, 826, 836, 846, 855, 864,
    /* ... continues for 256 points, one full sine cycle ... */
};
```

Annexure E: UART0 Driver, Receive Interrupt and Command Parsing

```
/* 115200 baud
void UART0_IRQHandler(void)
{
    uint32_t int_id = LPC_UART0->IIR;
    char c;
    if ((int_id & 0x0E) == 0x04) {
        c = LPC_UART0->RBR;
        if (c == '\r' || c == '\n') {
            if (rx_index > 0) {
                rx_buffer[rx_index] = '\0';
                command_ready = 1;
                rx_index = 0;
            }
        } else if (rx_index < 30) {
            rx_buffer[rx_index++] = c;
        }
    }
}

if (rx_buffer[0] == 'F' || rx_buffer[0] == 'f') {
    new_freq = atoi((const char *)&rx_buffer[1]);
    if (new_freq >= 10 && new_freq <= 15000) current_freq = new_freq;
} else if (rx_buffer[0] == 'W' || rx_buffer[0] == 'w') {
    w_type = rx_buffer[1];
    if (w_type == '1') current_waveform = WAVE_SINE;
} else if (rx_buffer[0] == 'A' || rx_buffer[0] == 'a') {
    new_amp = atoi((const char *)&rx_buffer[1]);
    if (new_amp >= 1 && new_amp <= 10) current_amplitude = new_amp;
}
```

MPP_FINAL_REPORT_D1_GROUP1

by Nagendra Kudva

Submission date: 25-Oct-2026 09:14AM (UTC+0800)
Submission ID: 3069173358
File name: MPP_FINAL_REPORT_D1_GROUP1.docx (1.87M)
Word count: 4157
Character count: 22268